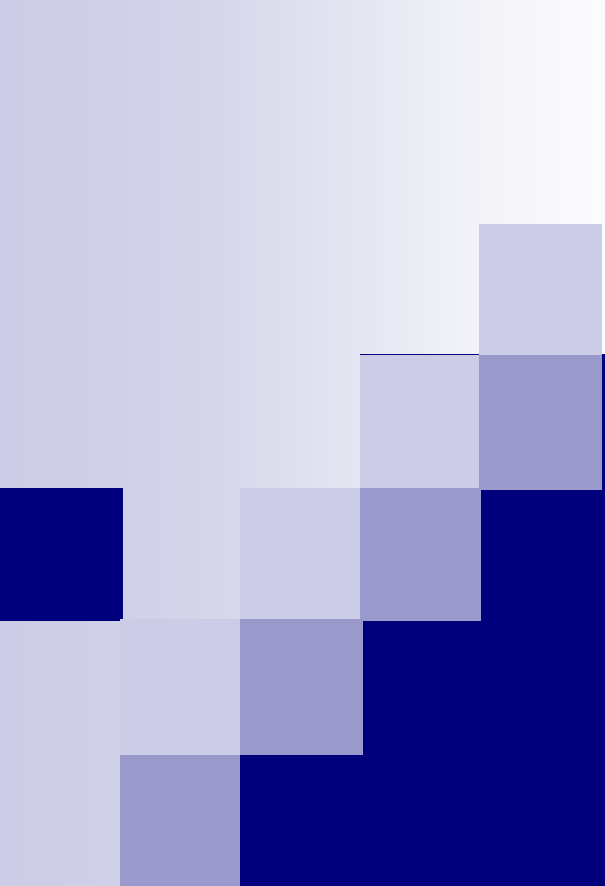




Modern Operating Systems

Chapter 5 – Disks

Zhang Yang
Spring 2025



Content of the Lecture

■ 5.4 Disks

- Disk Formatting
- Disk Arm Scheduling Algorithm
- Error Handling



Disk Formatting (1)

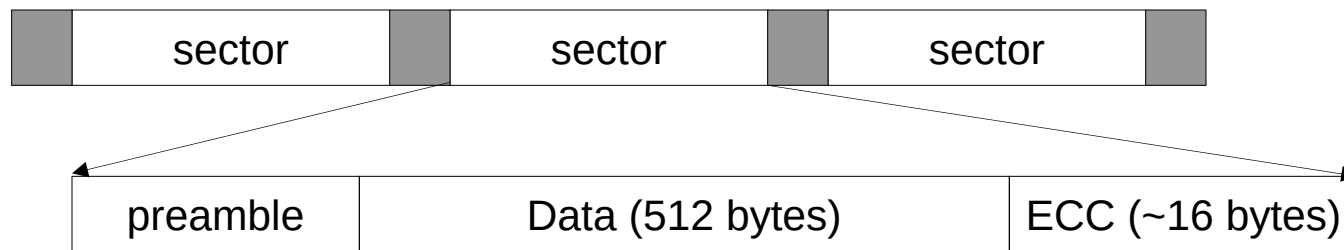
- After manufacturing, there is no information on the disk.
 - Just empty bits.
- Each platter need to have
 - A low-level format
 - A high-level formatbefore disk can be used.
- Low-level Formatting
 - Dividing a disk into sectors that the disk controller can read and write.
 - Low-level formatting is usually done by vendors

Disk Formatting (2)

■ Low-Level Formatting (ctd.)

□ Each track should have formatted in the following way:

- Sectors and inter-sector-gap between them.
- A sector will be have format like the following:
 - Preamble – data (512 bytes) – Checksum



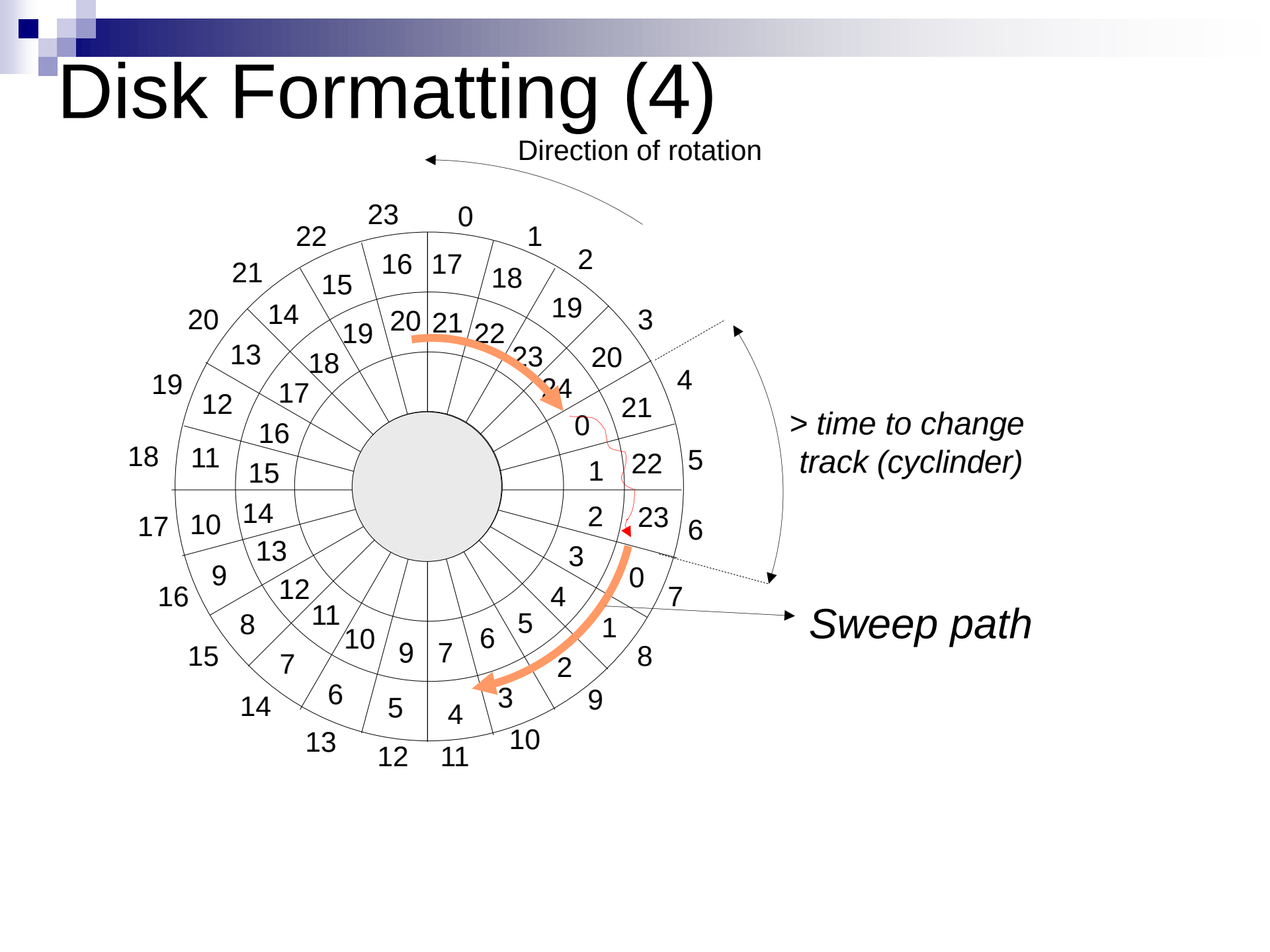
-certain bit pattern
- cylinder number
-sector number

Disk Formatting (3)

■ Low-Level Formatting (ctd.)

□ Cylinder Skew

- When reading sequential blocks, the seek time can result in missing block 0 in the next track.
- Disk can be formatted using a cylinder skew to avoid this.
- Cylinder Skew: Start cylinders at different points to give the head time to jump from one to the next.



Disk Formatting (5)

■ Low-Level Formatting (ctd.)

□ Cylinder Skew

- Example: 10000rpm disk drive, 300 sectors per track, seek time $800\mu\text{s}$, the cylinder skew is 40 sectors.

- 10000 rpm

- Drive rotates in 6 ms

- Track has 300 sectors

- New sector every $20\mu\text{s}$

- If track seek time $800\mu\text{s}$

- $\Rightarrow$ 40 sectors pass on seek

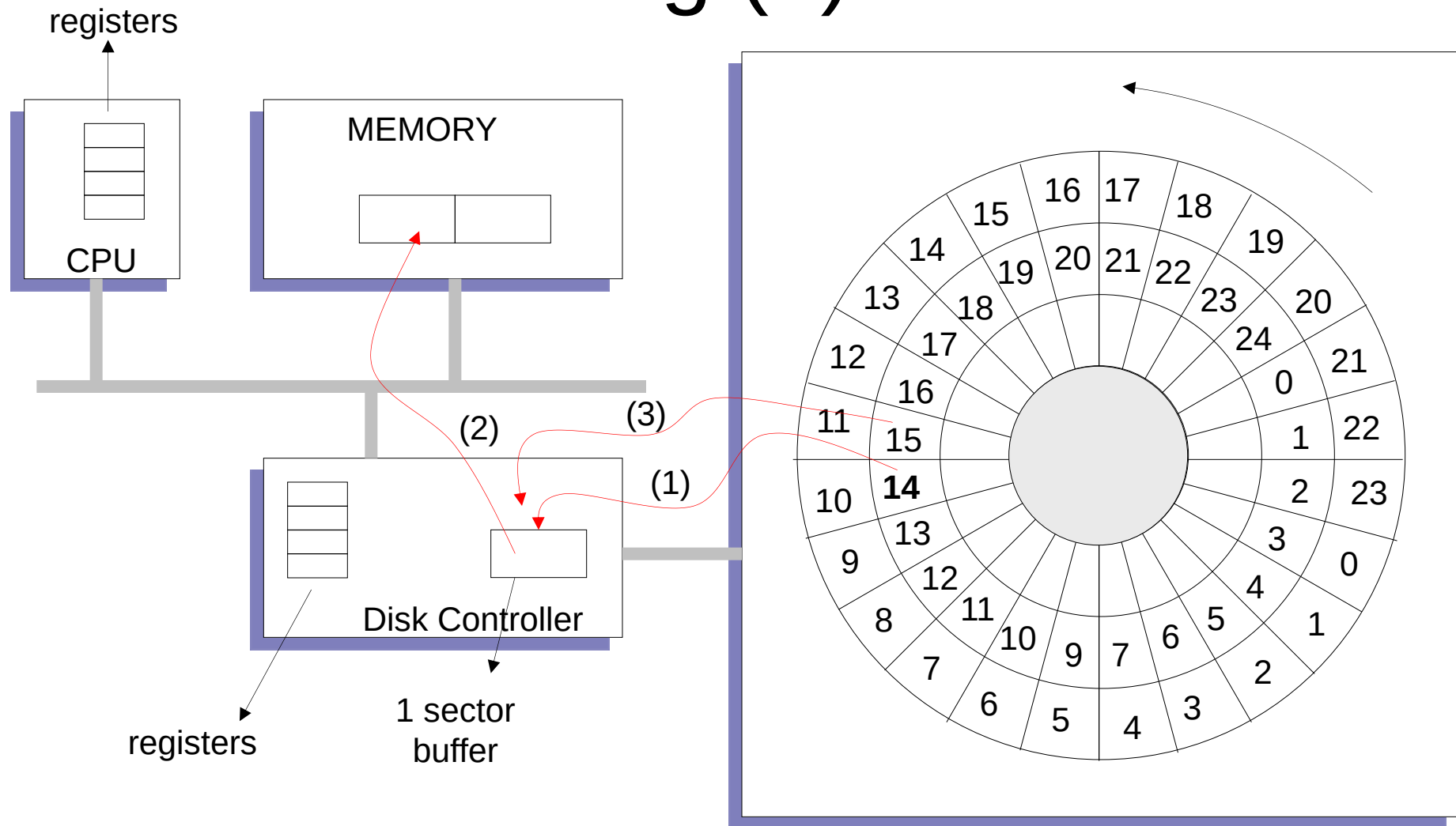
Disk Formatting (6)

■ Low-Level Formatting (ctd.)

□ Interleaving

- If we have ONE sector buffer in the disk controller, after we have transferred one sector of data from hard-disk to controller buffer:
 - We will copy the controller buffer to the memory.
 - During this time, the disk-head will pass the start of the next sector.
 - Therefore, the next logical sector, should not be the next physical sector in hard-disk.
 - There should be some interleaving.

Disk Formatting (7)

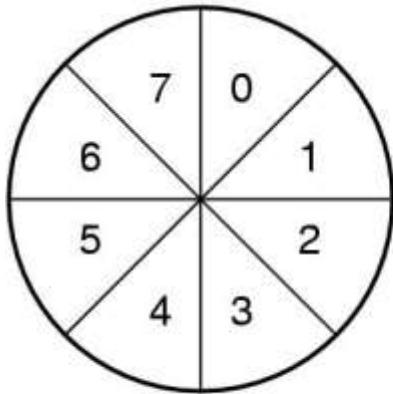


Disk Formatting (7)

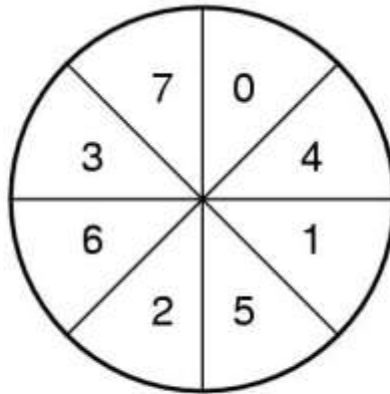
■ Low-Level Formatting (ctd.)

□ Interleaving (ctd.)

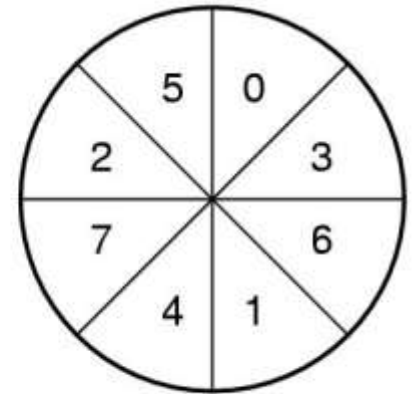
- (a) No interleaving
- (b) Single interleaving (more common)
 - Skipping one sector between reads.
- (c) Double interleaving
 - Skipping two sectors between reads.



(a)



(b)



(c)



Disk Formatting (8)

- Low-Level Formatting (ctd.)

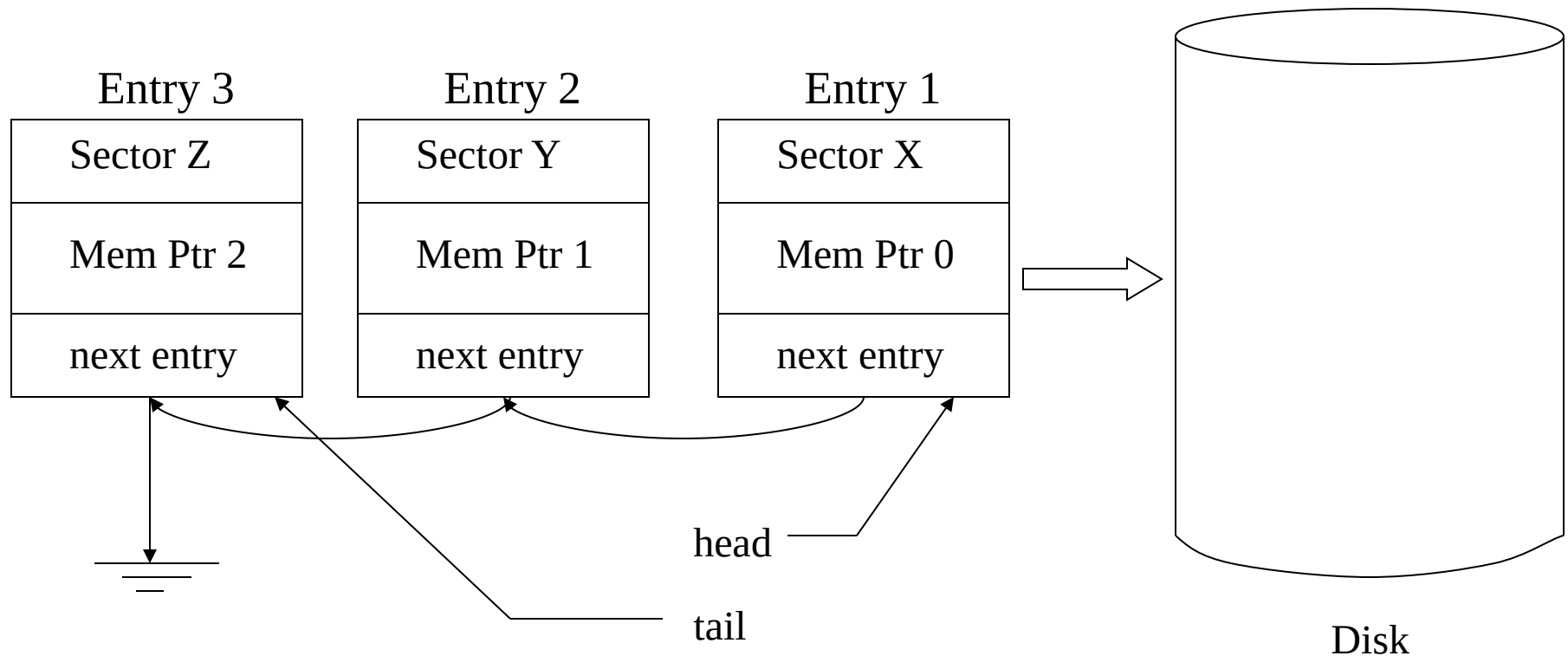
- Interleaving (ctd.)

- Modern drives overcome interleaving type issues by simply reading the entire track (or part thereof) into the on-disk controller and caching it.

Disk Queues (1)

- Each disk has a queue of jobs waiting to access disk.
 - Read jobs
 - Write jobs
- Each entry in queue contains the following:
 - Pointer to memory location to read/write from/to.
 - Sector number to access.
 - Pointer to next job in the queue.
- OS usually maintains this queue.

Disk Queues (2)



Disk Arm Scheduling (1)

- How long does it take to read a block (sector)?
 - Positioning Time + Transfer Time
 - Positioning Time
 - Seek Time
 - Move arm to the correct cylinder.
 - Rotational Delay
 - Wait until correct sector comes under head.
 - Actual data transfer time
- For most systems, the seek time dominates the other two times.

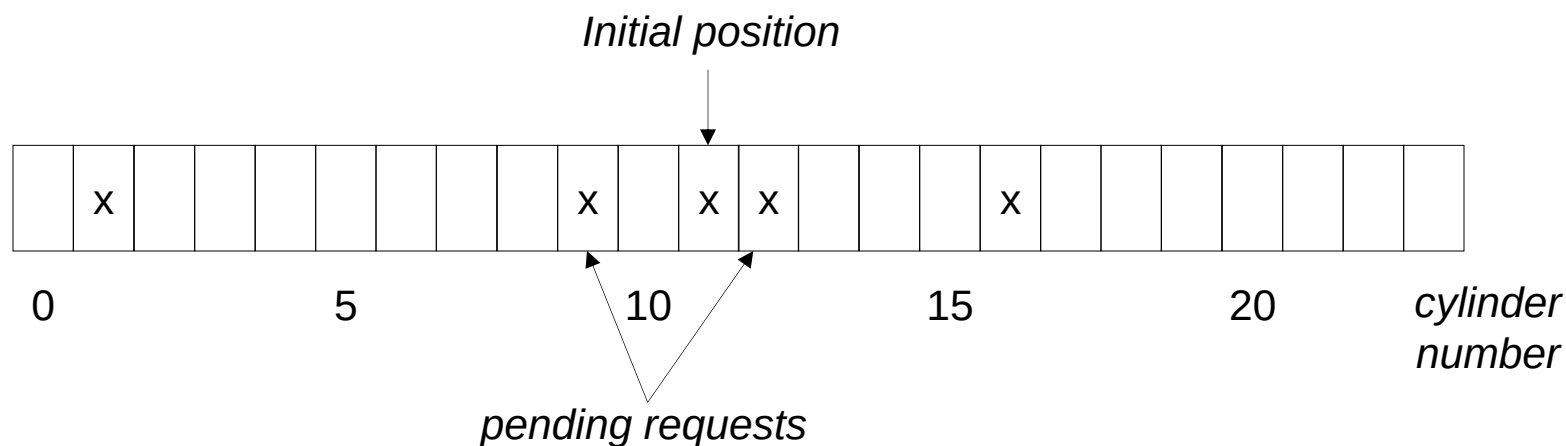
Disk Arm Scheduling (2)

- Assume the disk is heavily loaded.
 - There are a lot of requests for disk blocks that are arriving to the hard disk driver.
 - The drivers queues the requests.
 - The driver knows for each block request, where in hard disk the block should be stored (the cylinder number).
 - Since moving between cylinders (seek time) is costly, we should try to minimize these seek times.

Disk Arm Scheduling (3)

■ Disk Driver

- Keeps a table of requests.
- Table is indexed by the cylinder number.
- Blocks requests for the same cylinder are put in a linked list.



Disk Arm Scheduling Algorithms

- Goal: Minimize positioning time
 - Performed by both OS and disk itself.
- Several Algorithms Exist
 - First Come First Served (FCFS)
 - Advantage: Fair
 - Disadvantage: High seek cost and rotation
 - Shortest Seek First (SSF)
 - Advantage: Reduces arm movement (seek time)
 - Disadvantage: Unfair, can starve some requests
 - Elevator Algorithm (SCAN)
 - Advantage: More fair to requests, similar performance as SSTF
 - C-SCAN
 - Provides a more uniform wait time than SCAN.

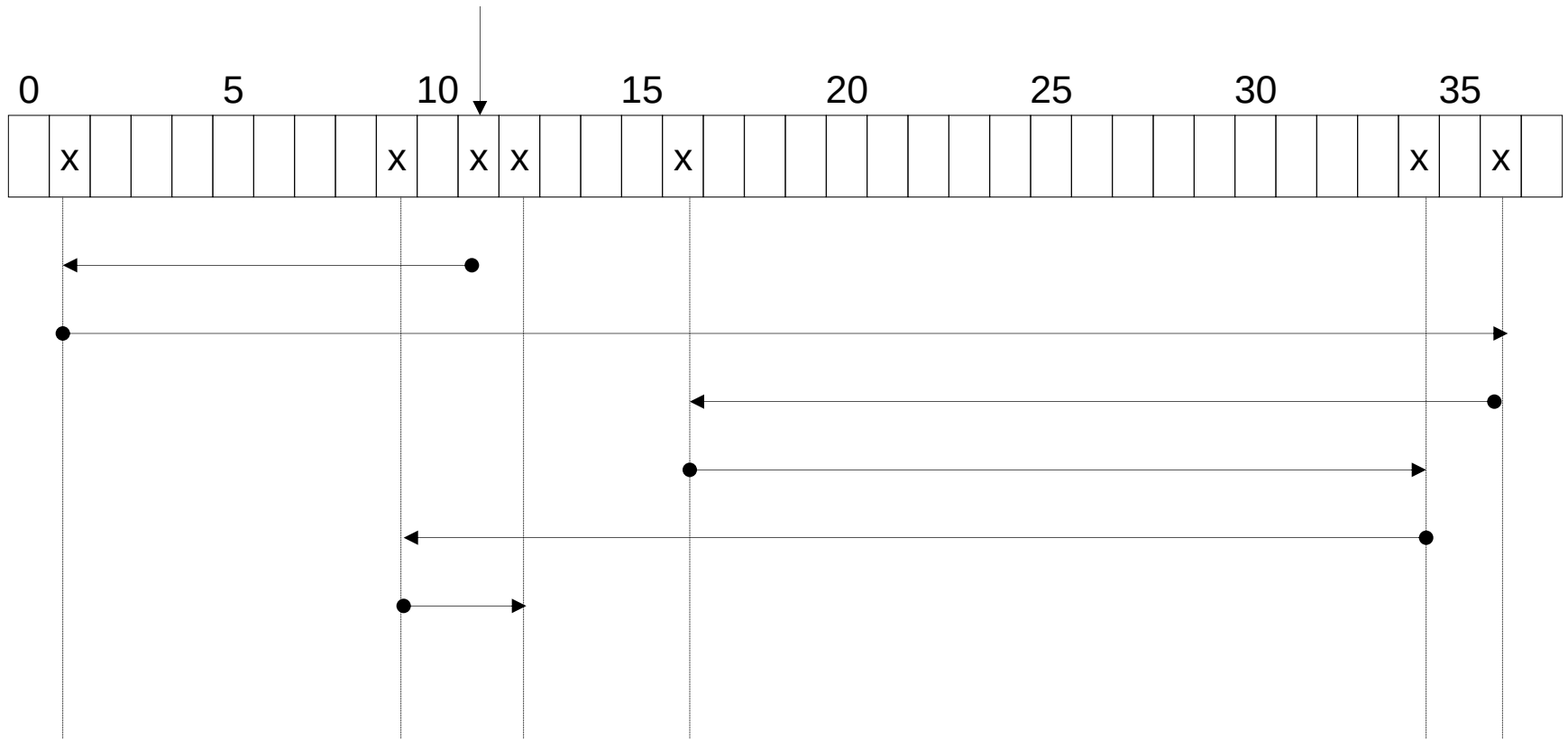
FCFS (1)

- Serve the request in the order they arrive (don't look to the cylinder numbers)
- Example
 - Assume the current position is cylinder 11.
 - The following requests arrive in the given time order:
1, 36, 16, 34, 9, 12
 - The request will be served in the same order.
 - The head will go to:
 - Cylinder 1 first (10 cylinder motion)
 - Cylinder 36 next (35 cylinder motion)
 - Cylinder 16 next (20)
 - Cylinder 34 next (18)
 - Cylinder 9 next (25)
 - Cylinder 12 next (3)
 - Total of 111 cylinders are skipped. Cost = 111

FCFS (2)

New request arrival sequence: 1, 36, 16, 34, 9, 12

Initial head position



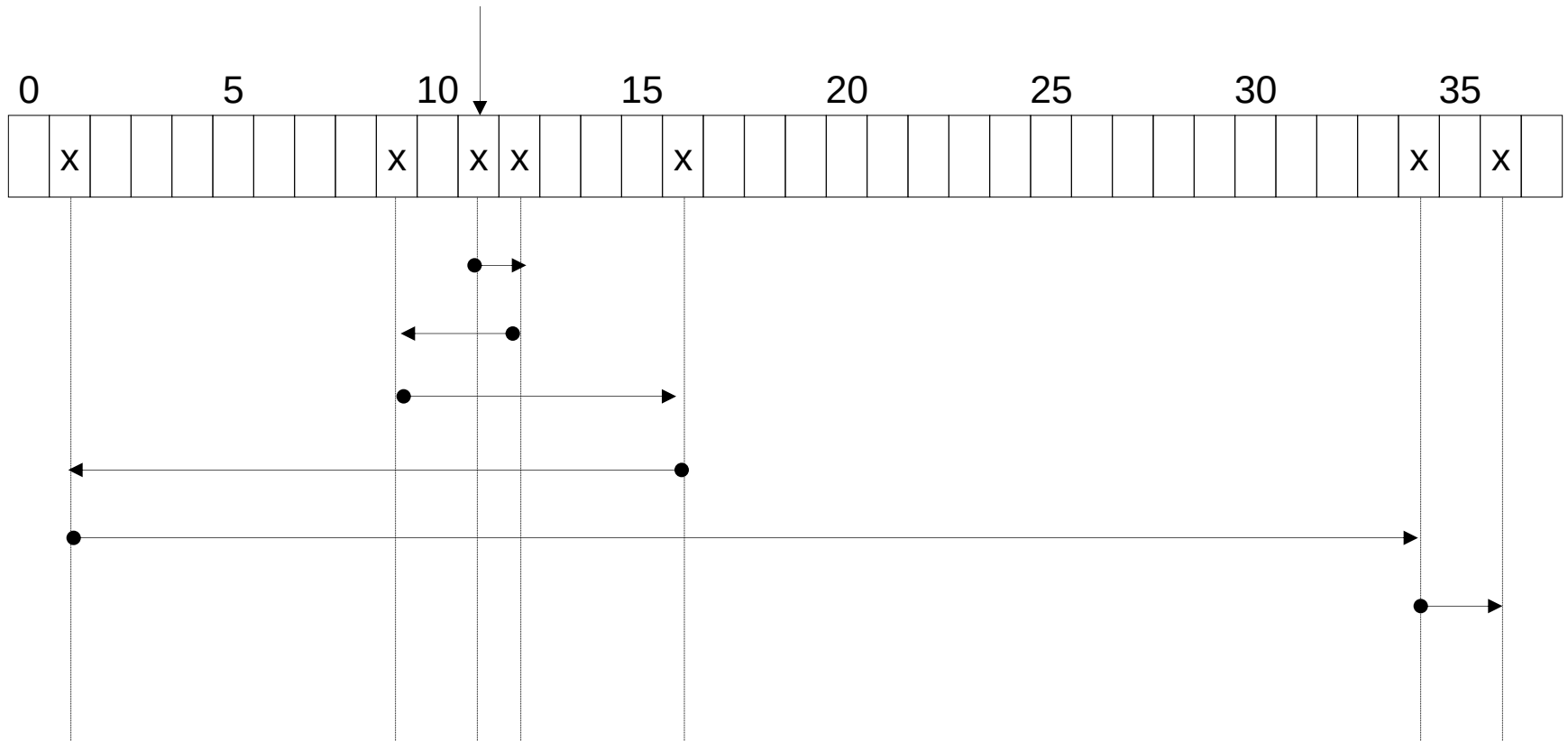
Shortest Seek First (SSF) (1)

- Satisfy the request that need shortest seek from the current position.
- In the previous example, this algorithms moved the head to cylinders in the order given below.
 - 12, 9, 16, 1, 34, and 36.
- Moving the head to these cylinders require:
 - 1, 3, 7, 15, 33, and 2 arm motions (that many cylinders are skipped at every request).
 - Total cost = 61.

Shortest Seek First (SSF) (2)

New request arrival sequence: 1, 36, 16, 34, 9, 12

Initial head position



Shortest Seek First (SSF) (3)

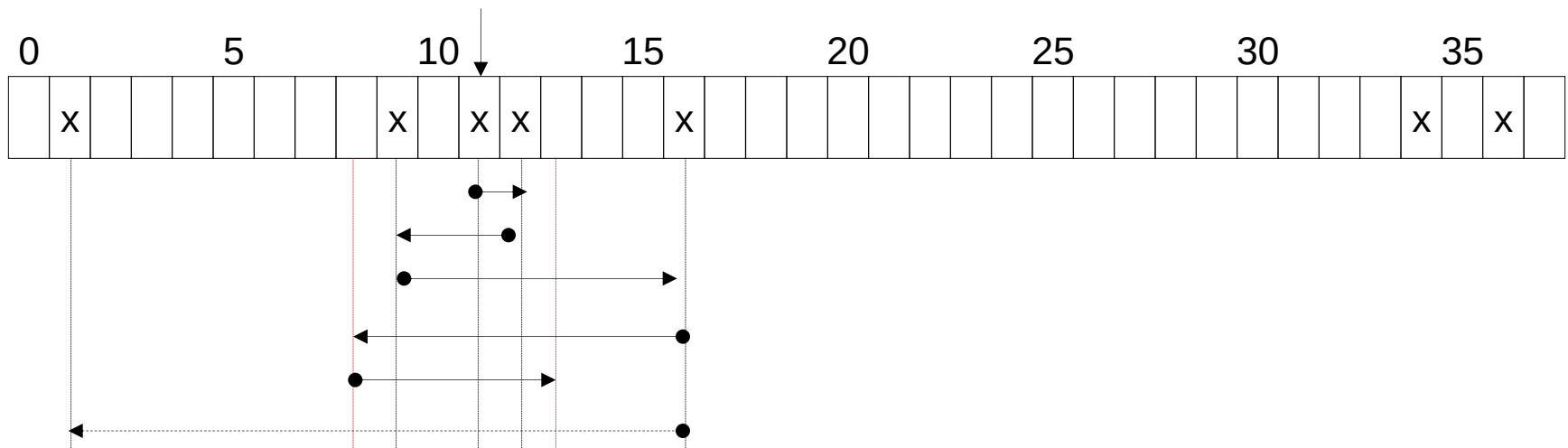
■ Issue

- The goals of minimal response time and fairness are in conflict here.
- Example

New request arrival sequence: 1, 36, 16, 34, 9, 12

After going to 16, a new request for 8 is present, then 13.

Initial head position



Elevator Algorithm (1)

- Also called SCAN algorithm.
- Do like what an elevator does.
 - 1. Move in one direction until all request in that direction is served
 - 2. Then change direction.
 - 3. Move in the new direction until all request in that direction are server.
 - Repeat steps 1, 2, and 3.
- In the previous example
 - Assume initial position was 11 and initial direction was up (going to higher numbered cylinders).

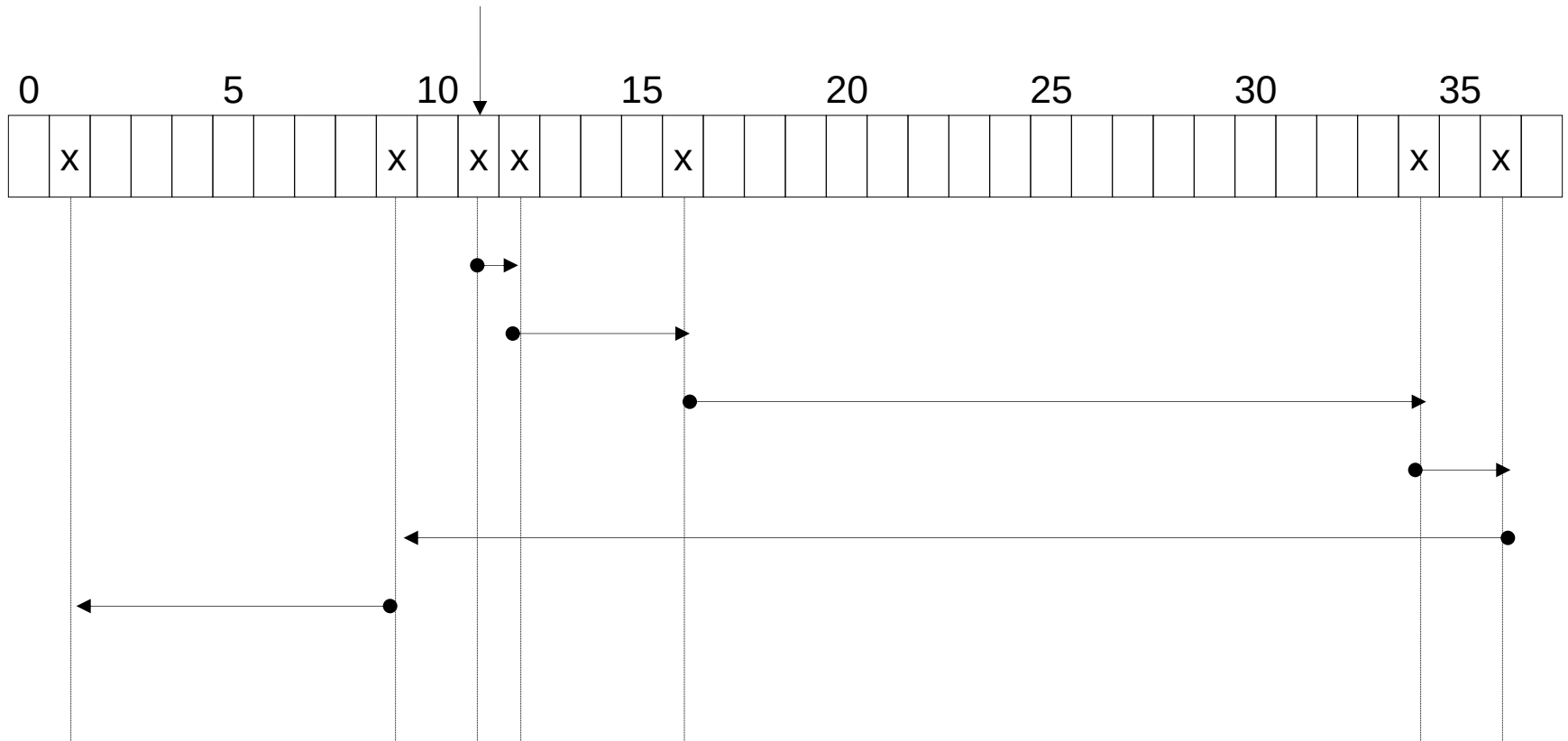
Elevator Algorithm (2)

- The request are server in the following order:
 - 12, 16, 34, 36, 9, 1
- Arm motions (# of cylinder skips)
 - 1, 4, 18, 2, 27, 8
 - Total cost = 60 cylinders.
- Performs good and is fair.
- Software should maintain 1 bit to know the direction. Elevator also should know whether it is going UP or DOWN

Elevator Algorithm (3)

New request arrival sequence: 1, 36, 16, 34, 9, 12

Initial head position



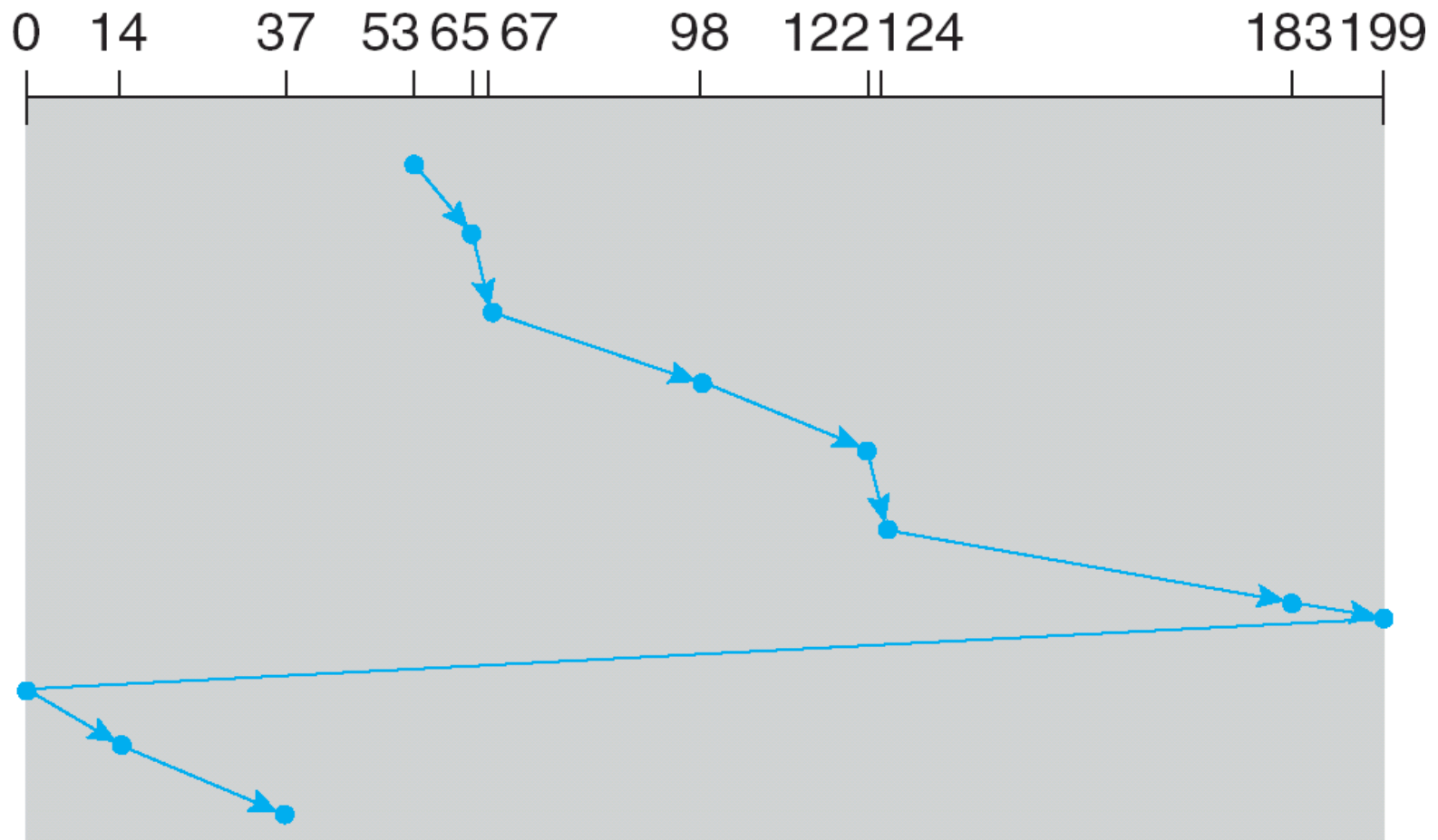
C-SCAN (1)

- Provides a more uniform wait time than SCAN.
- The head moves from one end of the disk to the other.
 - Servicing requests as it goes.
 - When it reaches the other end it immediately returns to beginning of the disk.
 - No requests serviced on the return trip.
- Treats the cylinders as a circular list.
 - That wraps around from the last cylinder to the first one.
- Circular SCAN

C-SCAN (2)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53





Selecting a Disk Scheduling Algorithm

- SSF is common and has a natural appeal.
- SCAN and C-SCAN perform better for systems that place heavy load on the disk.
- Performance depends on the number and types of requests.
- Requests for disk service are influenced by the file allocation method.

Error Handling (1)

- Disk defect cause bad blocks
- If the defect is small, let the ECC correct
- Otherwise, let entire sector go bad and deal with bad sector in controller or OS
- Two approaches to handle bad sectors
 - Deal with them in controller.
 - Disk tested before shipping.
 - Bad sector marked and substituted with spare.
 - Example (next page)
 - Deal with them in operating system.
 - OS check for bad sectors and record them.

Error Handling (2)

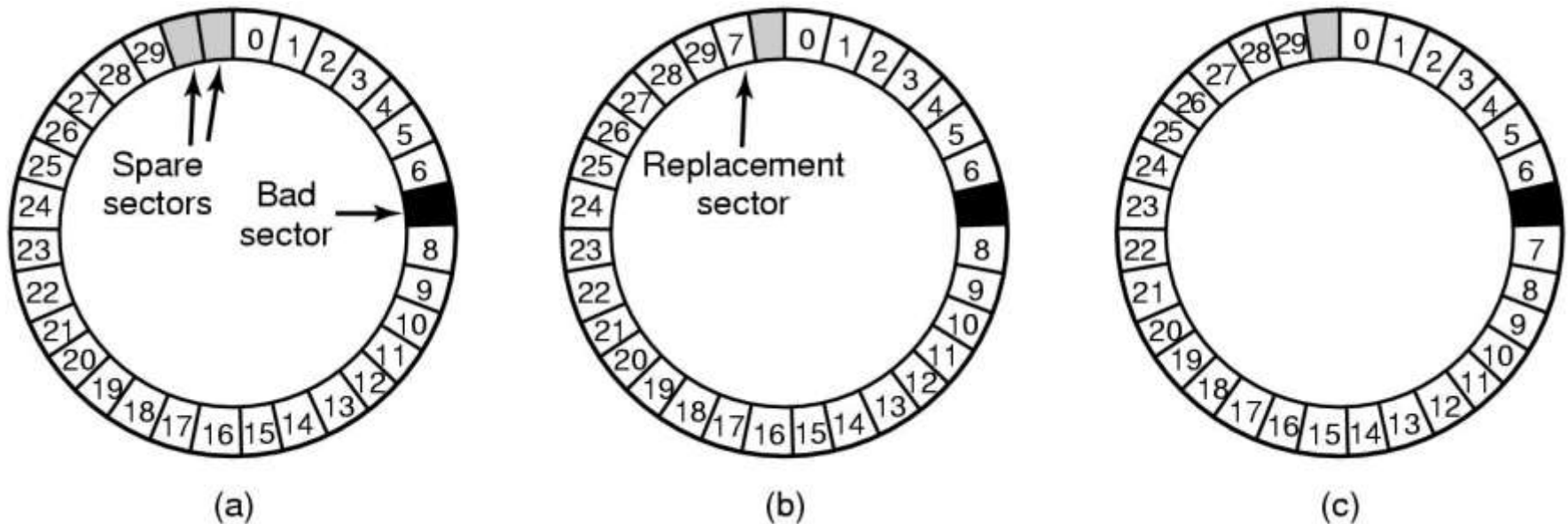


Figure 5-30

- (a) A disk track with a bad sector
- (b) Substituting a spare for the bad sector
- (c) Shifting all the sectors to bypass the bad one



Homework

- P431 27, 28, 31